

Code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define INITIAL_BUFFER_SIZE 16
6
7 typedef struct HistoryNode
8 {
9     char *command;
10    struct HistoryNode *prev;
11    struct HistoryNode *next;
12 } HistoryNode;
13
14 typedef struct
15 {
16     char *buffer;
17     size_t size;
18     size_t length;
19 } InputBuffer;
20
21 HistoryNode *head = NULL;
22 HistoryNode *tail = NULL;
23 HistoryNode *current = NULL;
24
25 /* Initialize input buffer */
26 void init_buffer(InputBuffer *input)
27 {
28     input->size = INITIAL_BUFFER_SIZE;
29     input->length = 0;
30
31     input->buffer = malloc(input->size);
32
33     if (input->buffer == NULL)
34     {
35         perror("malloc");
36         exit(EXIT_FAILURE);
37     }
38
39     input->buffer[0] = '\0';
40 }
41
42 /* Resize input buffer */
43 void resize_buffer(InputBuffer *input)
44 {
45     input->size *= 2;
46
47     char *temp = realloc(input->buffer, input->size);
48
49     if (temp == NULL)
50     {
51         free(input->buffer);
52         perror("realloc");
53         exit(EXIT_FAILURE);
54     }
55 }
```

```
55     input->buffer = temp;
56 }
57
58 /* Read input dynamically */
59 void read_input(InputBuffer *input)
60 {
61     int ch;
62
63     input->length = 0;
64     input->buffer[0] = '\0';
65
66     while ((ch = getchar()) != '\n' && ch != EOF)
67     {
68         if (input->length + 1 >= input->size)
69         {
70             {
71                 resize_buffer(input);
72             }
73
74             input->buffer[input->length++] = (char)ch;
75             input->buffer[input->length] = '\0';
76         }
77     }
78
79 /* Add command to history */
80 void add_history(const char *command)
81 {
82     HistoryNode *new_node;
83
84     new_node = malloc(sizeof(HistoryNode));
85
86     if (new_node == NULL)
87     {
88         perror("malloc");
89         exit(EXIT_FAILURE);
90     }
91
92     new_node->command = malloc(strlen(command) + 1);
93
94     if (new_node->command == NULL)
95     {
96         free(new_node);
97         perror("malloc");
98         exit(EXIT_FAILURE);
99     }
100
101     strcpy(new_node->command, command);
102
103     new_node->prev = tail;
104     new_node->next = NULL;
105
106     if (tail != NULL)
107     {
108 }
```

```

108     tail->next = new_node;
109 }
110 else
111 {
112     head = new_node;
113 }
114
115 tail = new_node;
116
117 current = NULL;
118 }
119
120 /* Display command history */
121 void show_history()
122 {
123     HistoryNode *temp = head;
124     int number = 1;
125
126     printf("\n\033[1;34mCommand History:\033[0m\n");
127
128     if (temp == NULL)
129     {
130         printf("No commands in history.\n");
131         return;
132     }
133
134     while (temp != NULL)
135     {
136         printf("%d %s\n", number, temp->command);
137
138         temp = temp->next;
139         number++;
140     }
141 }
142
143 /* Navigate to previous command */
144 char *previous_command()
145 {
146     if (tail == NULL)
147     {
148         printf("No command history.\n");
149         return NULL;
150     }
151
152     if (current == NULL)
153     {
154         current = tail;
155     }
156     else if (current->prev != NULL)
157     {
158         current = current->prev;
159     }
160     else

```

```

161     {
162         printf("Already at oldest command.\n");
163     }
164
165     return current->command;
166 }
167
168 /* Navigate to next command */
169 char *next_command()
170 {
171     if (current == NULL)
172     {
173         printf("Already at newest command.\n");
174         return NULL;
175     }
176
177     if (current->next != NULL)
178     {
179         current = current->next;
180         return current->command;
181     }
182
183     current = NULL;
184
185     printf("No newer command.\n");
186
187     return NULL;
188 }
189
190 /* Update input buffer */
191 void update_buffer(InputBuffer *input, const char *command)
192 {
193     size_t required = strlen(command) + 1;
194
195     while (input->size < required)
196     {
197         resize_buffer(input);
198     }
199
200     strcpy(input->buffer, command);
201
202     input->length = strlen(command);
203 }
204
205 /* Free command history */
206 void free_history()
207 {
208     HistoryNode *temp;
209
210     while (head != NULL)
211     {
212         temp = head;
213

```

```

214     head = head->next;
215
216     free(temp->command);
217     free(temp);
218 }
219
220 tail = NULL;
221 current = NULL;
222 }
223
224 /* Display memory information */
225 void display_buffer_info(InputBuffer *input)
226 {
227     printf("\n\033[1;33mInput Buffer Information:\033[0m\n");
228     printf("Buffer size   : %zu bytes\n", input->size);
229     printf("Current length: %zu bytes\n", input->length);
230 }
231
232 /* Main function */
233 int main()
234 {
235     InputBuffer input;
236
237     init_buffer(&input);
238
239     printf("\033[1;32m===== \033[0m\n");
240     printf("\033[1;32m Command History and Dynamic Buffer Demo \033[0m\n");
241     printf("\033[1;32m===== \033[0m\n");
242
243     printf("\nCommands:\n");
244     printf("  history  - Show command history\n");
245     printf("  prev     - Previous command\n");
246     printf("  next     - Next command\n");
247     printf("  buffer   - Show buffer information\n");
248     printf("  exit     - Exit program\n");
249
250     while (1)
251     {
252         printf("\n\033[1;36mshell> \033[0m");
253
254         read_input(&input);
255
256         if (strcmp(input.buffer, "exit") == 0)
257         {
258             break;
259         }
260
261         if (strcmp(input.buffer, "history") == 0)
262         {
263             show_history();
264             continue;
265         }

```

```

266     if (strcmp(input.buffer, "prev") == 0)
267     {
268         char *command = previous_command();
269
270         if (command != NULL)
271         {
272             update_buffer(&input, command);
273
274             printf("\033[1;35mRecalled: %s\033[0m\n",
275                 input.buffer);
276         }
277
278         continue;
279     }
280
281     if (strcmp(input.buffer, "next") == 0)
282     {
283         char *command = next_command();
284
285         if (command != NULL)
286         {
287             update_buffer(&input, command);
288
289             printf("\033[1;35mRecalled: %s\033[0m\n",
290                 input.buffer);
291         }
292
293         continue;
294     }
295
296     if (strcmp(input.buffer, "buffer") == 0)
297     {
298         display_buffer_info(&input);
299         continue;
300     }
301
302     if (strlen(input.buffer) > 0)
303     {
304         add_history(input.buffer);
305
306         printf("\033[1;32mCommand stored:\033[0m %s\n",
307             input.buffer);
308     }
309 }
310
311 printf("\n\033[1;33mReleasing memory...\033[0m\n");
312
313 free(input.buffer);
314
315 free_history();
316
317

```

```

318     printf("\033[1;32mAll dynamically allocated memory released.\033[0m\n");
319
320     return 0;
321 }
322

```

Output:

```
Practical14.c Practical15.c Practical16.c Practical17.c  
saisurendra@SaiSurendra:~$ gcc Skill3.c -o Skill3  
saisurendra@SaiSurendra:~$ ./Skill3
```

```
=====
Command History and Dynamic Buffer Demo
=====
```

Commands:

```
history  - Show command history
prev     - Previous command
next     - Next command
buffer   - Show buffer information
exit     - Exit program
```

```
shell> ls
```

```
Command stored: ls
```

```
shell> pwd
```

```
Command stored: pwd
```

```
shell> exit
```

```
Releasing memory...
```

```
All dynamically allocated memory released.
```